CONFIRMED TECHNOLOGY STACK

AI-Powered Smart Waste Management System with IoT Integration for Bope-Poddala Divisional Secretariat

University of Ruhuna | Bachelor of Computer Science - Department of Computer Science
Version 1.0 | Confirmed October 2026

1. Purpose

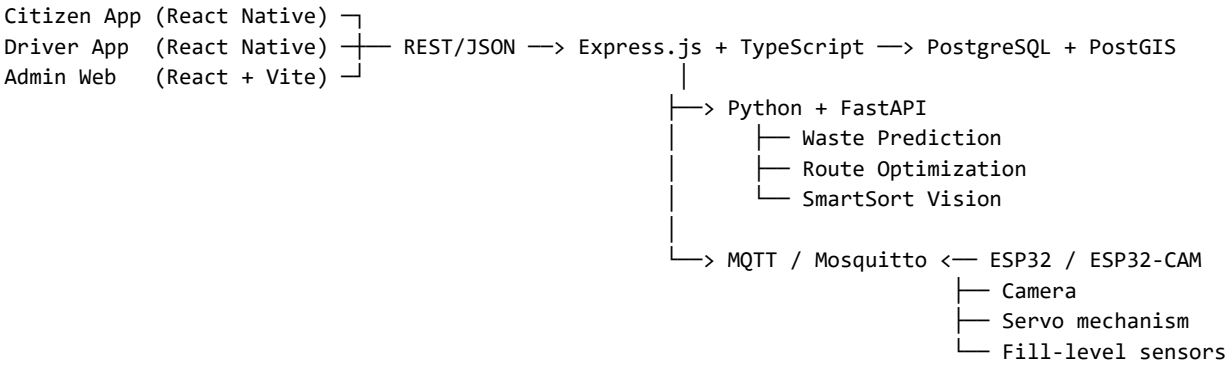
This document records the confirmed technology stack for the project. The stack is selected to support mobile and web applications, a TypeScript-based backend, geospatial waste-collection data, AI services, route optimization, and the SmartSort IoT prototype while remaining manageable for the five-member team.

2. Confirmed Technology Stack

System Area	Confirmed Technology	Role / Justification
Citizen Mobile App	React Native + TypeScript	Cross-platform citizen application.
Driver Mobile App	React Native + TypeScript	Cross-platform driver and collection application.
Admin Web Dashboard	React + TypeScript + Vite	Responsive administrative monitoring and management interface.
Backend API	Node.js + Express.js + TypeScript	Main REST API and business-logic layer.
Database	PostgreSQL + PostGIS	Relational storage plus geospatial support for collection points, GPS and routes.
ORM / Data Access	Prisma	Type-safe database access, migrations and schema management.
Authentication & Security	JWT + Argon2/bcrypt	Token-based authentication and secure password hashing.
AI Service Layer	Python + FastAPI	Independent APIs for prediction, optimization and vision inference.
Waste Prediction	pandas + scikit-learn / XGBoost	Data preparation, model training, evaluation and prediction.
Route Optimization	Python + OR-Tools	Optimization of waste-collection routes using collection and geographic data.
SmartSort Vision	Python + OpenCV + YOLO	Image preprocessing and controlled waste-category classification.
IoT Controller	ESP32 / ESP32-CAM	Camera/sensor control and SmartSort prototype communication.
IoT Messaging	MQTT	Lightweight telemetry and device messaging.
MQTT Broker	Eclipse Mosquitto	Broker for SmartSort device/backend communication.
Maps / Geospatial	OpenStreetMap + Leaflet (web)	Map visualization; mobile mapping library selected during implementation.
API Documentation	Swagger / OpenAPI	Interactive REST API documentation.
API Testing	Postman	Manual API development and integration testing.
UI/UX Design	Figma	Interface wireframes, prototypes and design collaboration.
Version Control	Git + GitHub	Source control, pull requests and team collaboration.

Project Management	Jira	Epics, stories/tasks, sprints, defects and progress tracking.
Deployment	Docker + Docker Compose	Repeatable deployment of backend, AI services, database and supporting services.

3. High-Level Architecture



4. Key Architecture Decisions

TypeScript across application layers: React Native, React and Express.js use TypeScript to reduce context switching and improve type safety.

PostgreSQL instead of MongoDB: The system contains strongly related data such as users, drivers, schedules, complaints, collection points and routes. PostGIS additionally supports geographic operations.

Separate Python AI services: AI and optimization remain in Python/FastAPI so the team can use the mature Python data-science, computer-vision and optimization ecosystems without mixing model logic into the Express backend.

Modular monolith backend: The Express backend should initially be one deployable application organized into clear modules rather than multiple business microservices.

MQTT for SmartSort telemetry: MQTT is used for lightweight IoT messaging between the SmartSort prototype and the server-side system.

Server-side SmartSort inference: ESP32/ESP32-CAM handles capture and device control while computationally heavier vision inference is performed by the Python service.

Docker Compose for deployment: The project does not require Kubernetes. Docker Compose is sufficient for an academic deployment and easier to operate before the December deadline.

5. Suggested Responsibility Alignment

Member	Primary Area	Main Technologies
Ishara Edirisinghe	Citizen Mobile App	React Native, TypeScript, API integration
Mahela Yapa	Driver App + Admin Dashboard	React Native, React, TypeScript, UI integration
Mohommed Rifaa	Waste Prediction AI	Python, pandas, scikit-learn/XGBoost, FastAPI
Chathuranga Gayashan	Route Optimization AI	Python, OR-Tools, PostGIS/geospatial integration
Ravindu Nadeesha	IoT + Coordination	ESP32/ESP32-CAM, MQTT, SmartSort hardware, integration coordination
Shared Team	Backend / DB / QA / Deployment	Express.js, PostgreSQL/PostGIS, Prisma, GitHub, Docker, testing

6. SmartSort Prototype Boundary

The SmartSort component remains a controlled prototype/public-station concept rather than a household-wide deployment. The initial classification categories are Plastic, Organic, Paper and Metal. The prototype should use controlled single-item input, fixed camera positioning and controlled lighting. Mixed, dirty or overlapping waste should not be presented as a guaranteed capability.

7. Development and Deployment Baseline

Core development, system integration, testing and deployment are targeted for completion by 31 December 2026. The period from 1-17 January 2027 is primarily reserved for report finalization, presentation/demo preparation, rehearsal and minor non-scope-changing fixes. The final presentation is scheduled for 18 January 2027.

8. Stack Control

This technology stack is treated as the project baseline. A major technology replacement should be made only when a clear technical blocker or supervisor-approved requirement justifies the change. Libraries within the selected ecosystems may be refined during implementation without changing the overall architecture.